

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4: Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL3*)
Assessment Tool Weightage: 30%

QUESTIONS: 15

Total Marks:30

Annexure A

CLASS TEST

Code of Conduct:

I D Varshini / 192320007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

CLASS TEST

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Annexure A

CLASS TEST

QUESTIONS:

Total Marks: 30

Q1. Define architectural patterns in object-oriented systems. List any two common patterns. Explain how the Model-View-Controller (MVC) architecture improves modularity and maintainability.

Answer: Architectural patterns are proven, reusable ways of organizing the major components of a software system and defining how those components interact. They provide a high-level structure that guides object responsibilities, communication, and separation of concerns. Two common architectural patterns are Model-View-Controller (MVC) and Layered Architecture. In MVC, the Model manages application data and business state, the View presents information to the user, and the Controller receives user input and coordinates actions between the View and Model. This separation improves modularity because changes to the user interface can usually be made without changing the business model, while changes to business logic need not affect every screen. It also improves maintainability, testing, parallel development, and reuse because each component has a focused responsibility and can evolve more independently.

[L1, L2, L3]

Q2. What is the Data Access Object (DAO) Pattern? Mention two advantages. Explain how DAO separates database-related operations from application logic.

Answer: The Data Access Object (DAO) pattern places all persistence operations for a particular type of data inside a dedicated object or interface. For example, a CustomerDAO may provide operations such as findById(), save(), update(), and delete(), while the service layer works with Customer objects rather than writing SQL directly. Two important advantages are: (1) separation of concerns, because database code is isolated from business code, and (2) easier maintenance and testing, because the data source or database technology can be changed with minimal effect on the rest of the application. DAO separates database-related operations by exposing a clear data-access interface. The application or business layer calls this interface, while the DAO implementation handles connections, queries, ORM calls, result conversion, and persistence details. Therefore, business logic remains focused on rules and use cases instead of database syntax.

[L2, L3, L4]

Q3.What are object-oriented design guidelines? Identify any two categories of design guidelines. Explain the role of the Liskov Substitution Principle (LSP) in developing reusable software components.

Answer: Object-oriented design guidelines are principles and recommendations used to create classes and collaborations that are understandable, reusable, extensible, and easy to maintain. Two useful categories are class-level guidelines, which deal with responsibilities, encapsulation, inheritance, and interfaces, and relationship/collaboration guidelines, which deal with coupling, dependency, composition, and communication among objects. The Liskov Substitution Principle (LSP) states that an object of a subtype should be usable wherever an object of its base type is expected without breaking the correctness of the program. A subclass must therefore preserve the meaningful behavior and contract of its parent. Following LSP leads to reusable software because client code can depend on abstractions and safely work with different implementations. It also prevents fragile inheritance hierarchies in which subclasses unexpectedly change assumptions made by the base class.

[L2, L3, L4]

Q4.Define database persistence in object-oriented applications. Mention two persistence mechanisms. Explain how an Object-Relational Mapping (ORM) framework supports persistent object management.

Answer: Database persistence is the ability of an object's state to survive after the program or object that created it has ended, by storing that state in durable storage and reconstructing it later. Two persistence mechanisms are relational database storage using SQL/ORM and object-oriented database storage in which objects are stored more directly. An Object-Relational Mapping (ORM) framework maps classes and object attributes to relational tables and columns. It converts operations on objects into database operations, manages object identity and relationships, and reconstructs objects from stored rows. ORM tools also commonly provide transactions, lazy/eager loading, change tracking, and query APIs. As a result, developers can manage persistent objects using object-oriented concepts instead of repeatedly writing low-level SQL and manually converting database rows into objects, which reduces boilerplate code and keeps persistence concerns more organized.

[L1, L2, L3]

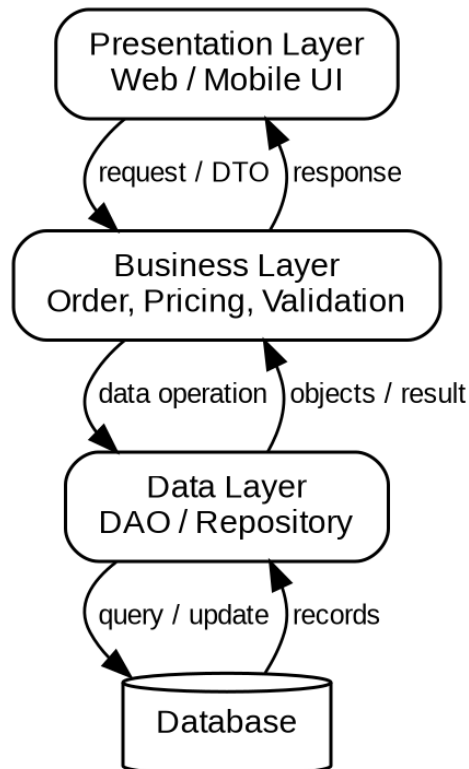
Q5.What is an Application Layer? State two major responsibilities. Explain how the Application Layer coordinates use cases without directly implementing domain business rules.

Answer: The Application Layer is the layer that exposes and coordinates the use cases of a system. It sits between the presentation layer and the domain/business layer and acts as an orchestration boundary. Two major responsibilities are: (1) receiving a use-case request and coordinating the required domain objects or services, and (2) managing application-level concerns such as transaction boundaries, authorization checks, input/output DTOs, and workflow sequencing. It does not directly implement core business rules. Instead, it asks domain objects and domain services to perform those rules. For example, in a fund-transfer use case, an application service may load the two accounts, call domain operations to validate and transfer the amount, and then ask a repository to save the changes. The rule determining whether a transfer is valid remains inside the domain layer, keeping business logic reusable and independent of the user interface.

[L2, L3, L4]

Q6.Explain the structure of a three-tier architecture consisting of Presentation, Business, and Data layers. Analyze the flow of a customer transaction through these layers in an e-commerce application.

Answer: A three-tier architecture separates a system into the Presentation Layer, Business Layer, and Data Layer. The Presentation Layer contains web pages, mobile screens, controllers, and input handling. The Business Layer contains business rules, validation, pricing, order processing, inventory checks, and transaction logic. The Data Layer contains repositories/DAOs that communicate with databases or other persistent stores. In an e-commerce purchase, the customer selects items and submits an order through the presentation layer. The request is passed to the business layer, which validates the cart, verifies stock, calculates price and tax, and coordinates payment/order rules. The business layer then asks the data layer to read or update product, customer, inventory, and order records. The data layer performs the database operations and returns results. Finally, the business layer builds the outcome and the presentation layer displays confirmation or an error. This structure limits responsibilities and makes each tier easier to test and change.



*Figure 6: Three-tier transaction flow
[L2, L4, L5]*

Q7.What is the role of a Business Logic Layer? List two important functions. Explain how it validates and processes requests received from the presentation layer.

Answer: The Business Logic Layer (BLL) contains the rules and decisions that define how the application behaves. It converts a user request into meaningful business operations rather than merely passing data between screens and the database. Two important functions are business validation and business processing/workflow coordination. When a request arrives from the presentation layer, the BLL first validates information according to domain rules, such as checking that an order quantity is positive, stock is available, or an account has sufficient balance. It then performs the required calculations and decisions, invokes domain objects or services, and requests persistence through repositories or DAOs. It returns a success result, calculated data, or a meaningful error to the presentation layer. Because validation and processing are centralized in the BLL, the same rules can be reused by a web UI, mobile app, or API without duplicating logic.

[L2, L3, L4]

Q8.Explain the significance of cohesion and coupling in object-oriented design. Analyze how high cohesion and low coupling contribute to software maintainability and reusability.

Answer: Cohesion describes how strongly the responsibilities inside a class or module are related, while coupling describes the degree of dependency between different classes or modules. High cohesion is desirable because a class should have a clear, focused purpose; for example, an InvoiceCalculator should mainly handle invoice calculations rather than also perform UI rendering and database access. Low coupling is desirable because classes should depend on small, stable interfaces rather than concrete internal details of other classes. Together, high cohesion and low coupling improve maintainability because a change is more likely to remain localized and less likely to create side effects elsewhere. They also improve reusability because a focused component with few external dependencies can be moved or used in another context more easily. Testing becomes simpler, replacement of implementations is easier, and the overall design becomes more understandable and adaptable to future requirements.

[L3, L4, L5]

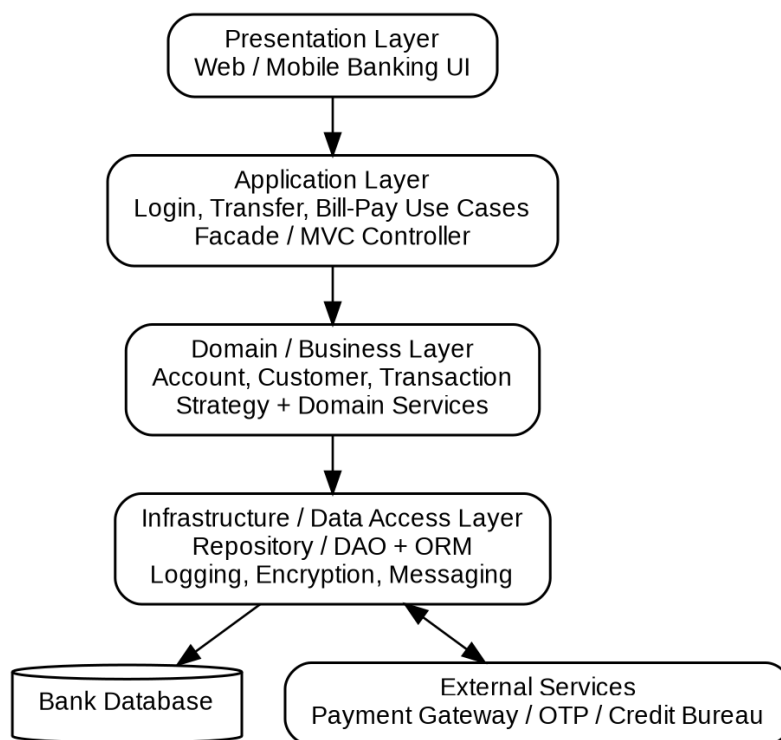
Q9.What is encapsulation in an object-oriented system? Explain how encapsulation protects object state and compare it with data hiding using a suitable example.

Answer: Encapsulation is the object-oriented practice of combining data and the operations that manipulate that data inside a class and exposing a controlled public interface. It protects object state by preventing external code from changing important attributes in arbitrary or invalid ways. For example, a BankAccount may keep balance as a private field and provide deposit() and withdraw() methods. The withdraw() method can reject negative amounts or withdrawals that violate account rules, so the object always protects its own valid state. Data hiding is closely related but focuses specifically on restricting direct access to implementation details, usually through private or protected members. Encapsulation is broader because it includes both hiding data and defining the behavior through which that data can be used. Thus, making balance private is data hiding, while placing balance together with controlled deposit/withdraw behavior inside BankAccount is encapsulation.

[L2, L4]

Q10.Design a layered object-oriented system for an online banking application. Identify the major layers, recommend suitable design patterns, and justify how the architecture improves security, scalability, and maintainability.

Answer: A suitable online banking system can use four main layers: Presentation, Application, Domain/Business, and Infrastructure/Data Access. The Presentation Layer contains secure web/mobile views and controllers. The Application Layer coordinates use cases such as login, balance enquiry, beneficiary management, transfer, and bill payment. The Domain Layer contains entities such as Customer, Account, Beneficiary, and Transaction together with core banking rules. The Infrastructure Layer handles repositories/DAOs, ORM, database access, logging, encryption, messaging, and external payment/OTP services. MVC can organize the UI, Facade/Application Service can expose use cases, Repository or DAO can isolate persistence, Strategy can support different payment/interest rules, and Dependency Injection can connect implementations through interfaces. Security improves because authentication, authorization, validation, encryption, and auditing can be enforced at clear boundaries. Scalability improves because layers/services can be scaled or replaced independently. Maintainability improves because business rules, UI code, and technical infrastructure are separated, reducing ripple effects when one part changes.



*Figure 10: Layered architecture for online banking
[L3, L4, L5]*

Q11.What is the role of the Infrastructure Layer in object-oriented architecture? List two responsibilities. Explain how it supports technical services such as database access and external system communication.

Answer: The Infrastructure Layer provides the technical capabilities required by the higher layers of an object-oriented application. It contains implementation details that are important for operation but are not themselves core business rules. Two common responsibilities are persistence/database access and integration with external systems or platform services. For database access, the infrastructure layer implements repository or DAO interfaces, manages ORM/database connections, transactions, queries, and mapping between stored data and domain objects. For external communication, it may contain adapters or clients for payment gateways, email/SMS services, cloud storage, REST APIs, or message brokers. Higher layers depend on abstractions, while the infrastructure layer supplies the concrete implementations. This keeps the domain model independent of a particular database, network protocol, or vendor, making the system easier to test and allowing technical technologies to be changed with less impact on business code.

[L1, L2, L3]

Q12.Define the Singleton Design Pattern. Mention its major characteristics. Explain its appropriate use in managing shared resources and discuss one limitation of the pattern.

Answer: The Singleton Design Pattern ensures that a class has only one controlled instance and provides a global access point to that instance. Its major characteristics are a private or restricted constructor, a static reference to the single instance, and a public method or property that returns that instance. It can be appropriate for a resource that truly needs one shared coordinator, such as an application-wide configuration registry or a single logging manager. Centralizing such a resource can avoid repeated initialization and provide consistent access. However, Singleton should be used carefully. A major limitation is that it introduces global state and hidden dependency, which can make unit testing and parallel execution harder. It may also become a bottleneck or create unnecessary coupling if many classes directly access it. In modern designs, dependency injection is often preferred when a shared service does not strictly require Singleton semantics.

[L2, L3, L4]

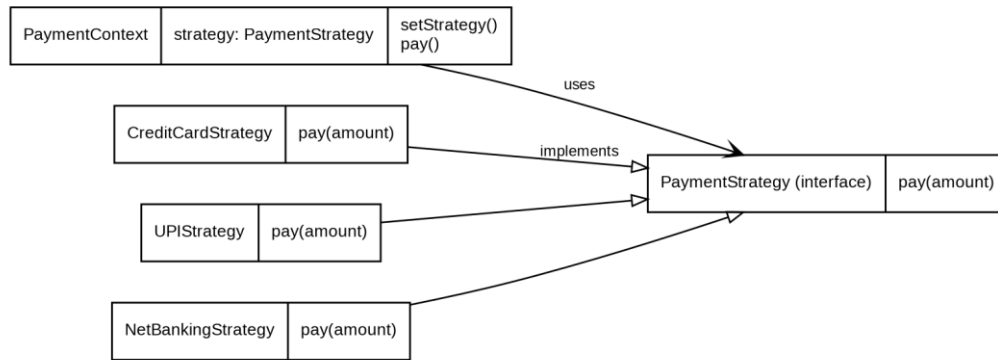
Q13.What is dependency injection in object-oriented design? List two benefits. Analyze how dependency injection reduces tight coupling between classes and improves testability.

Answer: Dependency Injection (DI) is a design technique in which an object receives the other objects or services it depends on from outside rather than creating those dependencies internally. Dependencies are commonly supplied through a constructor, setter, or DI container. Two benefits are reduced coupling and improved testability. For example, an OrderService can depend on a PaymentGateway interface that is passed through its constructor instead of directly executing new RealPaymentGateway(). The service now knows only the interface, so the real gateway can be replaced by another implementation without changing OrderService. This reduces tight coupling to a particular class or technology. During unit testing, a fake or mock PaymentGateway can be injected to simulate success, failure, or edge cases without contacting a real payment provider. DI therefore supports substitution, clearer dependencies, easier automated testing, and more maintainable object-oriented architectures.

[L2, L3, L4]

Q14.Explain the working of the Strategy Design Pattern. Compare the roles of Context and Strategy with a suitable real-world application example.

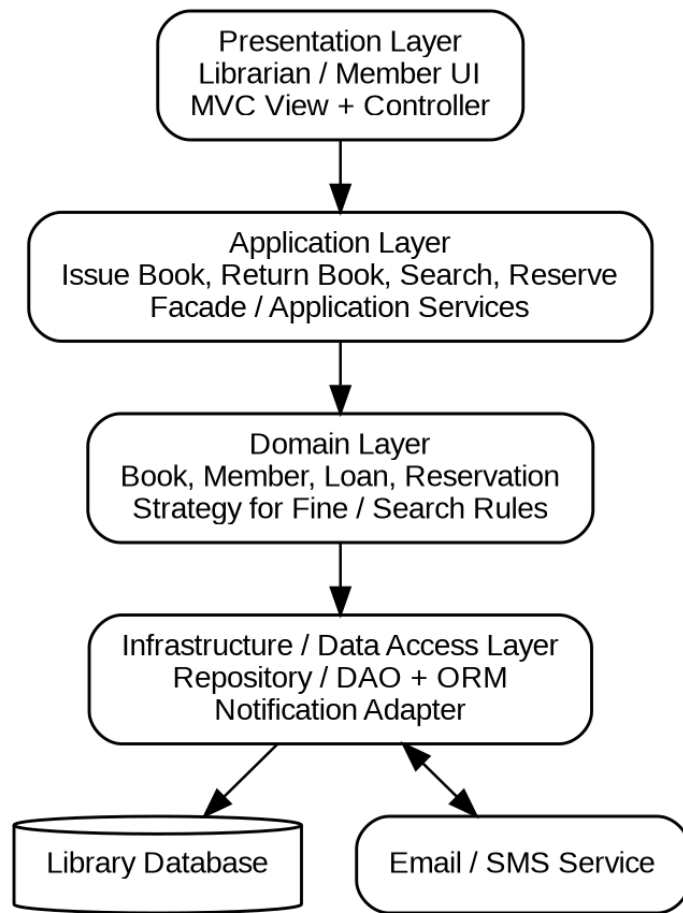
Answer: The Strategy Design Pattern defines a family of interchangeable algorithms behind a common interface and allows the algorithm to be selected or changed at runtime. The Strategy declares the operation that all algorithms must provide, while Concrete Strategies implement different versions of that operation. The Context is the object that needs the behavior; it stores a reference to a Strategy and delegates the work to that strategy rather than containing every algorithm itself. For example, an online checkout can use a PaymentStrategy interface with CreditCardStrategy, UPIStrategy, and NetBankingStrategy implementations. PaymentContext receives the strategy chosen by the customer and calls pay(amount). The Context controls when payment is required, while the Strategy controls how that payment is performed. This design avoids large conditional statements, follows the Open/Closed Principle, and makes new payment methods easier to add without modifying the checkout workflow.



*Figure 14: UML class view of the Strategy pattern
[L3, L4, L5]*

Q15.Design a layered architecture for a Library Management System. Identify suitable layers, recommend relevant design patterns, and justify how your proposed architecture supports flexibility and maintainability.

Answer: A Library Management System can be organized into Presentation, Application, Domain, and Infrastructure/Data Access layers. The Presentation Layer provides librarian and member screens for search, issue, return, reservation, and account operations. The Application Layer coordinates use cases such as IssueBook, ReturnBook, RenewLoan, ReserveBook, and RegisterMember. The Domain Layer contains core objects such as Book, BookCopy, Member, Loan, Reservation, and Fine with rules about availability, borrowing limits, due dates, and penalties. The Infrastructure Layer implements repositories/DAOs, ORM/database access, and notification adapters for email or SMS. MVC can structure the user interface, Facade/Application Service can provide use-case entry points, Repository/DAO can separate persistence, Strategy can support different fine or search rules, and Dependency Injection can connect components through interfaces. Flexibility improves because new databases, notification methods, or rule strategies can be introduced with limited changes. Maintainability improves through clear responsibilities, low coupling, high cohesion, and independent testing of layers.



*Figure 15: Layered architecture for Library Management System
[L3, L4, L5]*